Library Database Application

Mini Project – CMPT 354 – Database Systems I

(1) The Team

Group 52 (Trent Carlson, Harry Kim)

(2) Project Specification

Below are the defined details of our project, based on the information provided:

Library has print books, online books, magazines, scientific journals, CDs, records, etc.

- Define an Items schema, which will allow people to find items by their title, item type, or a uniquely identifying item ID key.

People can borrow the items from library and return by the due date.

- Define a People schema, with their name and a uniquely identifying person ID key.
- Define a Borrowings schema, which will allow people to borrow an item, record the borrow date, assign a due date, and record when the item was returned. Borrowings will be attached to a single person and a single item. An item will be unable to be borrowed if there's already a Borrowing that hasn't been returned. Borrowings will also have a uniquely identifying borrowing ID.

People may be subject to fines if they do not return items by the due date.

- Define a Fines schema, with a uniquely identifying fine ID key, the amount of the fine, and the status of payment. Stores a borrowing ID as a foreign key referencing Borrowings, since a fine is connected to some borrowing. Fines may be issued in the amounts of $5, $15, or $50, for varying lateness or for a damaged item.

Library also holds book clubs, book related events, art shows, film screenings, etc.

- Define an Events schema, which will record the event's name, description, and event date. Events will be uniquely identified by an event ID key.

Library events are recommended for specific audiences.

- Define an Audiences schema, which will specify the type of audience, and be uniquely identified through an audience ID key.
- Define an EventAudiences schema, which will record what audiences an event is recommended for. Stores the event ID and audience ID as foreign keys referencing Events and Audiences respectively.
- Define a PersonAudiences schema, which will record what type of audiences a person belongs to, representing a many-to-one relationship, as one person can be part of many audience types. Stores the person ID and audience ID as foreign keys referencing People and Audiences respectively.

Library events are held on library social rooms.

- Define a SocialRooms schema, which will record a room's name and a uniquely identifying social_room_ID.
- Add a foreign key referencing SocialRooms to the Events schema, representing that an event is held in a specific social room.

People can attend library events for free.

- Define an Attending schema, which will record the event ID and person ID as foreign keys referencing Events and People respectively, so that what people are attending an event can be tracked. People will be able to sign up to an event.

Library also has personnel and record keeping for personnel.

- Define a Personnel schema, which inherits the attributes of a Person, but with an additional attribute denoting the personnel's role.
- Add a personnel ID attribute as a foreign key referencing Person to the Borrowings schema for tracking what personnel helped a person to borrow an item.

Library also keeps records of items (books, etc.) that might be added to library in the future.

- Define a FutureItems schema, which, like an Item, will have an item type and a title, with a uniquely identifying candidate item ID.

Ask for help from a librarian

- Define a HelpRequests schema, which will allow people to submit a request for help to a specified personnel. A help request has a uniquely identifying request ID, a status, the message contents, and is associated with a person ID as a foreign key.

Database Schema

Primary keys are underlined, foreign keys are superscripted:

Items = { item_id, item_type, title }

FutureItems = { candidate_item_id, item_type, title }

People = { person_id, name }

Personnel = { person_id(FK-People), role }

Borrowings = { borrowing_id, item_id(FK-Items), person_id(FK-People), personnel_id(FK-Personnel), borrow_date, due_date, return_date }

Fines = { fine_id, borrowing_id(FK-Borrowings), amount, paid_status }

Events = { event_id, name, description, event_date, room_id(FK-SocialRooms) }

Attending = { event_id(FK-Events), person_id(FK-People) }

SocialRooms = { room_id, room_name }

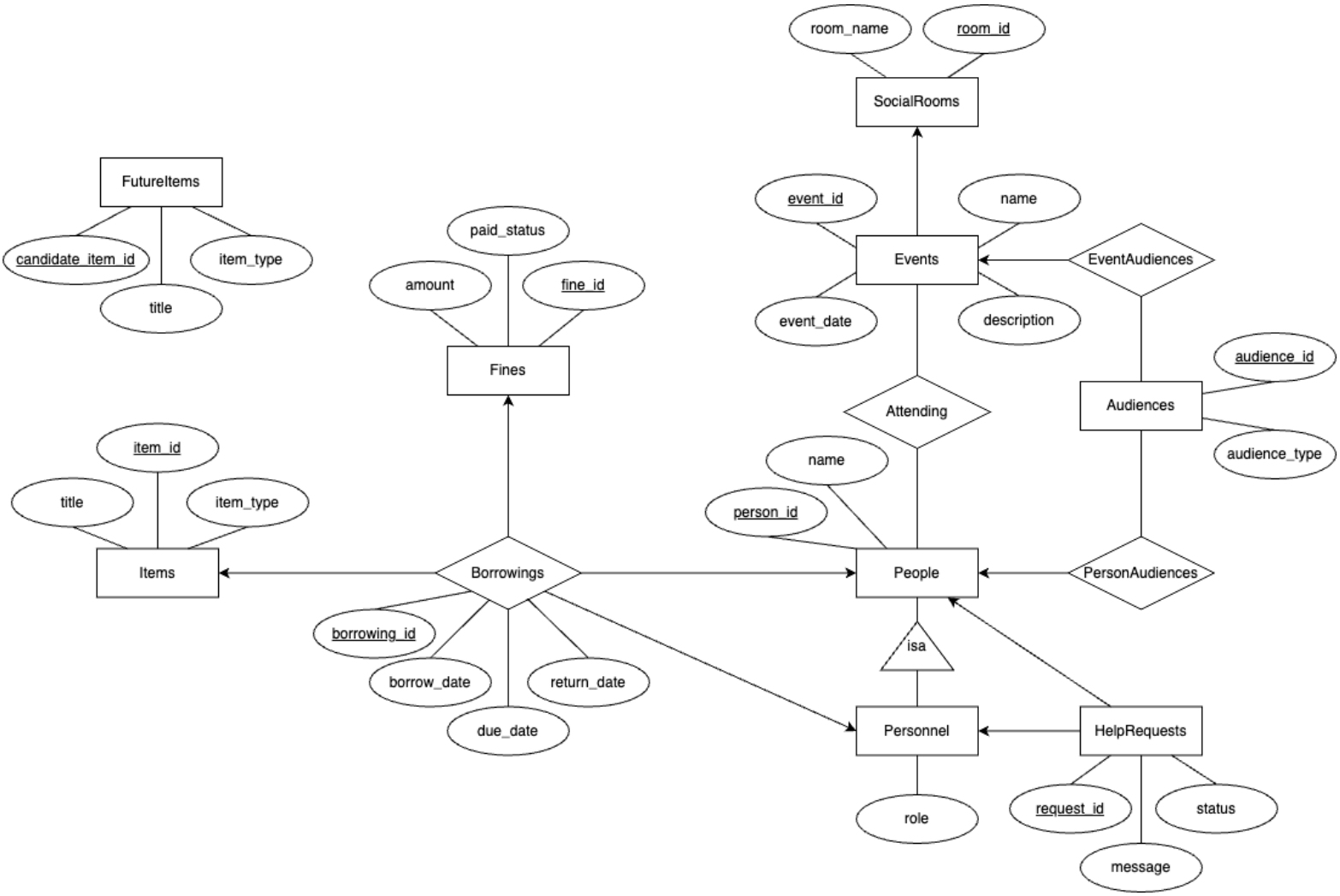
Audiences = { audience_id, audience_type }

EventAudiences = { event_id(FK-Events), audience_id(FK-Audiences) }

PersonAudiences = { person_id(FK-People), audience_id(FK-Audiences) }

HelpRequests = { request_id, person_id(FK-People), personnel_id(FK-Personnel) message }

(3) E/R Diagram


room_name
room_id
SocialRooms
FutureItems
candidate_item_id
item_type
title
paid_status
amount
fine_id
Fines
event_id
name
Events
EventAudiences
event_date
description
audience_id
Audiences
audience_type
Attending
item_id
title
item_type
Items
name
person_id
Borrowings
People
PersonAudiences
borrowing_id
borrow_date
return_date
due_date
isa
Personnel
HelpRequests
role
request_id
status
message

(4) Does the design allow anomalies?

Functional Dependencies (FDs)

Items

- item_id → item_type, title
- Since each item has a unique identifier, it logically determines the type and title. This ensures each item is accurately represented, and even multiple copies of the same item can be tracked individually.

FutureItems

- candidate_item_id → item_type, title
- Similar to Items, this FD ensures that proposed new items are uniquely identifiable and distinguishable from existing ones. This prevents confusion between new items and ones that are already registered in the system.

People

- person_id → name
- Similar to Items, uniquely identifying every person by their ID ensures that names are associated with the correct individual, even in circumstances like having the same name.

Personnel

- person_id → role
- As personnel have roles as well as names, this should also be uniquely determined by their ID. This preserves data integrity and ensures there is no redundant records.

Borrowings

- borrowing_id → borrow_date, due_date, return_date, item_id, person_id, personnel_id
- This FD shows that all information connected to a Borrowing is uniquely identified by its ID. As borrowing_id determines item_id, person_id, and personnel_id, it can be used to find the associated borrower, item, and personnel.

Fines

- fine_id → amount, paid_status, borrowing_id
- This ensures that fines are properly tracked and linked to their corresponding borrowings.

Events

- event_id → name, description, event_date, room_id
- Events are uniquely identified by event_id, and has a connected room_id representing where it is taking place.

Attending

- No non-trivial functional dependencies.

SocialRooms

- room_id → room_name
- Each room has a unique room_id, eliminating ambiguity with room assignments.

Audiences

- audience_id → audience_type
- This ensures every audience category is distinct and correctly identified.

EventAudiences

- No non-trivial functional dependencies.

PersonAudiences

- No non-trivial functional dependencies.

HelpRequests

- request_id $\rightarrow$ message, status, person_id, personnel_id
- Each request has a unique request_id, to avoid any collisions from requests if their attributes are identical.

Our design does not allow anomalies. This is because the schema is in BCNF as for all non-trivial functional dependencies X $\rightarrow$ Y that X is a superkey. Thanks to decomposition, most of the determinants of these FDs are primary keys, meaning they are candidate keys. There are also no partial keys thanks to this.

(5) SQL Schema

```
-- Items table
CREATE TABLE Items (
    item_id INTEGER PRIMARY KEY,
    item_type TEXT,
    title TEXT,
    CHECK (item_type IN ('Book', 'DVD', 'Scientific Journal', 'Audiobook', 'Magazine',
'Newspaper', 'eBook', 'CD', 'Record', 'Video Game'));
);

-- FutureItems table
CREATE TABLE FutureItems (
    candidate_item_id INTEGER PRIMARY KEY,
    item_type TEXT,
    title TEXT,
    CHECK (item_type IN ('Book', 'DVD', 'Scientific Journal', 'Audiobook', 'Magazine',
'Newspaper', 'eBook', 'CD', 'Record', 'Video Game'));
);

-- People table
CREATE TABLE People (
    person_id INTEGER PRIMARY KEY,
    name TEXT
);

-- Borrowings table
CREATE TABLE Borrowings (
    borrowing_id INTEGER PRIMARY KEY,
    item_id INTEGER,
    person_id INTEGER,
    personnel_id INTEGER,
    borrow_date TEXT,
    due_date TEXT,
    return_date TEXT,
    FOREIGN KEY (item_id) REFERENCES Items(item_id),
    FOREIGN KEY (person_id) REFERENCES People(person_id),
    FOREIGN KEY (personnel_id) REFERENCES Personnel(person_id),
    CHECK (return_date IS NULL OR return_date >= borrow_date), -- Ensure return_date
is after borrow_date
    CHECK (due_date IS NULL OR due_date >= borrow_date) -- Ensure due_date is after
borrow_date
);
```

```sql
-- Trigger to prevent double borrowing
CREATE TRIGGER trg_prevent_double_borrowing
BEFORE INSERT ON Borrowings
FOR EACH ROW
WHEN NEW.return_date IS NULL
BEGIN
    SELECT
        CASE
            WHEN EXISTS (SELECT 1 FROM Borrowings WHERE item_id = NEW.item_id AND
return_date IS NULL)
            THEN RAISE(ABORT, 'This item is already borrowed and not yet returned.')
        END;
END;

-- Fines table
CREATE TABLE Fines (
    fine_id INTEGER PRIMARY KEY,
    borrowing_id INTEGER,
    amount REAL,
    paid_status INTEGER,
    FOREIGN KEY (borrowing_id) REFERENCES Borrowings(borrowing_id),
    CHECK (paid_status IN (0, 1)), -- 0 for unpaid, 1 for paid
    CHECK (amount IN (5, 15, 50)) -- Fixed fine amounts
);

-- SocialRooms table
CREATE TABLE SocialRooms (
    room_id INTEGER PRIMARY KEY,
    room_name TEXT
);

-- Events table
CREATE TABLE Events (
    event_id INTEGER PRIMARY KEY,
    name TEXT,
    description TEXT,
    event_date TEXT,
    room_id INTEGER,
    FOREIGN KEY (room_id) REFERENCES SocialRooms(room_id)
);
```

```sql
-- Attending table
CREATE TABLE Attending (
    event_id INTEGER,
    person_id INTEGER,
    PRIMARY KEY (event_id, person_id), -- Composite primary key
    FOREIGN KEY (event_id) REFERENCES Events(event_id),
    FOREIGN KEY (person_id) REFERENCES People(person_id)
);

-- Audiences table
CREATE TABLE Audiences (
    audience_id INTEGER PRIMARY KEY,
    audience_type TEXT
);

-- EventAudiences table
CREATE TABLE EventAudiences (
    event_id INTEGER,
    audience_id INTEGER,
    PRIMARY KEY (event_id, audience_id), -- Composite primary key
    FOREIGN KEY (event_id) REFERENCES Events(event_id),
    FOREIGN KEY (audience_id) REFERENCES Audiences(audience_id)
);

-- PersonAudiences table
CREATE TABLE PersonAudiences (
    person_id INTEGER,
    audience_id INTEGER,
    PRIMARY KEY (person_id, audience_id),
    FOREIGN KEY (person_id) REFERENCES People(person_id),
    FOREIGN KEY (audience_id) REFERENCES Audiences(audience_id)
);

-- Personnel table
CREATE TABLE Personnel (
    person_id INTEGER PRIMARY KEY,
    role TEXT,
    FOREIGN KEY (person_id) REFERENCES People(person_id),
    CHECK (role IN ('Librarian', 'Assistant Librarian', 'Volunteer')) -- Ensure role
is one of the specified values
);
```

```sql
-- Help requests table
CREATE TABLE HelpRequests (
    request_id INTEGER PRIMARY KEY,
    person_id INTEGER,
    personnel_id INTEGER,
    message TEXT,
    status INTEGER,
    FOREIGN KEY (person_id) REFERENCES People(person_id),
    FOREIGN KEY (personnel_id) REFERENCES Personnel(person_id),
    CHECK (status IN (0, 1)) -- Ensure status is either 'Open' or 'Closed' (0 for
Open, 1 for Closed)
);
```

(6) Populate Tables

In *library.db*, each table was populated with realistic tuples in the following quantities:

Table Name	Tuple Count
Items	50
FutureItems	10
People	20
Borrowings	20
Fines	10
SocialRooms	10
Events	10
Attending	10
Audiences	10
EventAudiences	10
PersonAudiences	10
Personnel	10
HelpRequests	10

(7) Database Application

Our database application is a web application using Flask. Library users can:

- Find an item in the library
- Borrow an item from the library
- Return a borrowed item
- Donate an item to the library
- Find an event in the library
- Register for an event in the library
- Volunteer for the library
- Ask for help from a librarian
- View borrowings & fines
- View future items
- Renew an item
- Pay a fine
- View & update help requests
- Add a new person
- Manage people

Ensure that you have Flask installed before running the application. Flask can be installed by running “pip install flask” in the Terminal. Then, the application can be started by running “python app.py” and visiting http://127.0.0.1:5000/. If that doesn’t work, end the program with CTRL-C and try running “flask run -h localhost -p 3000” and visiting http://localhost:3000/ instead.